

1. Transforming into Structured Summary

Transform these notes into a structured summary

Emotion Detection from Speech Project
voice signal collected from dataset
different audio formats wav mp3 converted into wav
sampling rate 16000 Hz used
noise removal preprocessing normalization done
convert audio to mel spectrogram
mfcc features extracted around 13 coefficients
also used chroma and zero crossing rate
pitch energy brightness considered
Show more

I'll create a structured summary of your emotion detection project right away!

Created a file, read a file

Done! I've organized your notes into a clear, structured summary with sections covering data preprocessing, feature extraction, model architecture, training details, challenges, and future work. The document flows logically from data collection through to future improvements.

Emotion detection summary
Document · MD

Download

Emotion detection summary · MD

Copy

Emotion Detection from Speech Project

Overview

A machine learning project that detects emotions from voice signals using LSTM neural networks.

Data Collection & Preprocessing

Audio Data

- **Source:** Voice signals from dataset
- **Format Conversion:** MP3 → WAV
- **Sampling Rate:** 16,000 Hz

Preprocessing Steps

- Noise removal
- Normalization
- Conversion to mel spectrogram

Feature Extraction

Features Used

- **MFCC** (Mel-Frequency Cepstral Coefficients): ~13 coefficients
- **Chroma:** Pitch class representation



Emotion detection summary · MD

Copy



Output

- Numerical values converted into sequence vectors
- Used as input for the model

Model Architecture

Network Type

- **Bidirectional LSTM** (forward and backward)

Output Layer

- Softmax activation for classification

Emotion Classes

- Happy
- Sad
- Angry
- Neutral
- Fear

Training Configuration

Data Split

- Training: 80%
- Testing: 20%



Emotion detection summary · MD

Copy



Training Configuration

Data Split

- Training: 80%
- Testing: 20%

Model Parameters

- **Loss Function:** Categorical cross-entropy
- **Optimizer:** Adam

Performance

- **Accuracy:** ~85%

Challenges Encountered

1. Background noise interference
2. Model overfitting
3. Dataset imbalance across emotion classes

Future Improvements

- Implement transformer-based models
- Expand with larger datasets
- Develop real-time emotion detection capability

2. Text generation

Text Generation

Share

Write a 300-word article on the importance of Artificial Intelligence in modern healthcare.

The Transformative Role of Artificial Intelligence in Modern Healthcare

Artificial Intelligence has emerged as a revolutionary force in healthcare, fundamentally changing how medical professionals diagnose diseases, treat patients, and manage health systems. As healthcare costs rise and populations age, AI offers unprecedented opportunities to improve outcomes while enhancing efficiency.

One of AI's most significant contributions is in diagnostic accuracy. Machine learning algorithms can analyze medical images—from X-rays to MRIs—with remarkable precision, often detecting abnormalities that human eyes might miss. In radiology and pathology, AI systems help identify early-stage cancers, cardiovascular diseases, and neurological conditions, enabling earlier interventions that can save lives. These tools don't replace physicians but rather augment their capabilities, allowing them to make more informed decisions.

AI also personalizes treatment through precision medicine. By analyzing vast datasets including genetic information, medical histories, and treatment outcomes, AI systems can

Text Generation

Share

physicians but rather augment their capabilities, allowing them to make more informed decisions.

AI also personalizes treatment through precision medicine. By analyzing vast datasets including genetic information, medical histories, and treatment outcomes, AI systems can predict which therapies will be most effective for individual patients. This approach minimizes trial-and-error in treatment selection and reduces adverse drug reactions, making healthcare both safer and more effective.

Beyond direct patient care, AI streamlines administrative operations that have long burdened healthcare systems. Natural language processing automates documentation, reducing the time clinicians spend on paperwork. Predictive analytics optimize hospital staffing, equipment allocation, and patient flow, preventing bottlenecks and improving resource utilization.

Drug discovery has also been accelerated by AI, with algorithms identifying promising compounds and predicting their effectiveness far faster than traditional methods. This capability became particularly evident during the COVID-19 pandemic, when AI assisted in vaccine development and treatment protocols.

However, implementing AI in healthcare requires careful attention to data privacy, algorithmic bias, and the preservation of the human element in medicine. When deployed responsibly, AI doesn't diminish the importance of compassionate care—it frees healthcare providers to focus more on their patients while delivering unprecedented precision and efficiency in medical practice.

3. Coding

Coding

Write a C program to implement AES encryption using a simple example.

Share

I'll create a C program that implements AES-128 encryption with a clear example. This implementation will show the core AES algorithm structure.

Creating a C program that implements AES-128 encryption with a working example >

Now let me compile and run the program to demonstrate it works:

Compiling and running the AES encryption program >

Perfect! Now let me move the file to the outputs directory and share it with you:

Ran a command, read a file >

I've created a complete C implementation of AES-128 encryption. The program includes:

Key Components:

- **S-box:** Substitution table for the SubBytes transformation
- **SubBytes:** Substitutes each byte using the S-box
- **ShiftRows:** Cyclically shifts rows in the state array
- **MixColumns:** Mixes data within each column using Galois field arithmetic

Coding

- **ShiftRows:** Cyclically shifts rows in the state array
- **MixColumns:** Mixes data within each column using Galois field arithmetic
- **AddRoundKey:** XORs the state with the round key
- **Key Expansion:** Generates 11 round keys from the original key

How it works:

1. The plaintext (16 bytes) goes through an initial round key addition
2. Then 9 main rounds of: SubBytes → ShiftRows → MixColumns → AddRoundKey
3. A final round without MixColumns
4. The result is the encrypted ciphertext

The example encrypts the test vector from the NIST FIPS-197 specification, demonstrating correct AES-128 implementation. You can modify the `plaintext` and `key` arrays in `main()` to encrypt different data.

To compile and run: `gcc aes_encryption.c -o aes && ./aes`

Aes encryption

Download

```
1 #include <stdio.h>
2 #include <stdint.h>
3 #include <string.h>
4
5 // AES-128 constants
6 #define AES_BLOCK_SIZE 16
7 #define AES_KEY_SIZE 16
8 #define NUM_ROUNDS 10
9
10 // S-box lookup table for SubBytes transformation
11 static const uint8_t sbox[256] = {
12     0x63, 0x7c, 0x77, 0x7b, 0xf2, 0x6b, 0x6f, 0xc5, 0x30, 0x01, 0x67, 0x2b, 0xfe, 0xd7, 0xab, 0x76,
13     0xca, 0x82, 0xc9, 0x7d, 0xfa, 0x59, 0x47, 0xf0, 0xad, 0xd4, 0xa2, 0xaf, 0x9c, 0xa4, 0x72, 0xc0,
14     0xb7, 0xfd, 0x93, 0x26, 0x36, 0x3f, 0xf7, 0xcc, 0x34, 0xa5, 0xe5, 0xf1, 0x71, 0xd8, 0x31, 0x15,
15     0x04, 0xc7, 0x23, 0xc3, 0x18, 0x96, 0x05, 0x9a, 0x07, 0x12, 0x80, 0xe2, 0xeb, 0x27, 0xb2, 0x75,
16     0x09, 0x83, 0x2c, 0x1a, 0x1b, 0x6e, 0x5a, 0xa0, 0x52, 0x3b, 0xd6, 0xb3, 0x29, 0xe3, 0x2f, 0x84,
17     0x53, 0xd1, 0x00, 0xed, 0x20, 0xfc, 0xb1, 0x5b, 0x6a, 0xcb, 0xbe, 0x39, 0x4a, 0x4c, 0x58, 0xcf,
18     0xd0, 0xef, 0xaa, 0xfb, 0x43, 0x4d, 0x33, 0x85, 0x45, 0xf9, 0x02, 0x7f, 0x50, 0x3c, 0x9f, 0xa8,
19     0x51, 0xa3, 0x40, 0x8f, 0x92, 0x9d, 0x38, 0xf5, 0xbc, 0xb6, 0xda, 0x21, 0x10, 0xff, 0xf3, 0xd2,
20     0xcd, 0x0c, 0x13, 0xec, 0x5f, 0x97, 0x44, 0x17, 0xc4, 0xa7, 0x7e, 0x3d, 0x64, 0x5d, 0x19, 0x73,
21     0x60, 0x81, 0x4f, 0xdc, 0x22, 0x2a, 0x90, 0x88, 0x46, 0xee, 0xb8, 0x14, 0xde, 0x5e, 0x0b, 0xdb,
22     0xe0, 0x32, 0x3a, 0x0a, 0x49, 0x06, 0x24, 0x5c, 0xc2, 0xd3, 0xac, 0x62, 0x91, 0x95, 0xe4, 0x79,
23     0xe7, 0xc8, 0x37, 0x6d, 0x8d, 0xd5, 0x4e, 0xa9, 0x6c, 0x56, 0xf4, 0xea, 0x65, 0x7a, 0xae, 0x08,
24     0xba, 0x78, 0x25, 0x2e, 0x1c, 0xa6, 0xb4, 0xc6, 0xe8, 0xdd, 0x74, 0x1f, 0x4b, 0xbd, 0x8b, 0x8a,
25     0x70, 0x3e, 0xb5, 0x66, 0x48, 0x03, 0xf6, 0x0e, 0x61, 0x35, 0x57, 0xb9, 0x86, 0xc1, 0x1d, 0x9e,
26     0xe1, 0xf8, 0x98, 0x11, 0x69, 0xd9, 0x8e, 0x94, 0x9b, 0x1e, 0x87, 0xe9, 0xce, 0x55, 0x28, 0xdf,
27     0x8c, 0xa1, 0x89, 0x0d, 0xbf, 0xe6, 0x42, 0x68, 0x41, 0x99, 0x2d, 0x0f, 0xb0, 0x54, 0xbb, 0x16
28 };
29
30 // Round constant for key expansion
```

```
29
30 // Round constant for key expansion
31 static const uint8_t rcon[11] = {
32     0x00, 0x01, 0x02, 0x04, 0x08, 0x10, 0x20, 0x40, 0x80, 0x1b, 0x36
33 };
34
35 // Function to perform SubBytes transformation
36 void sub_bytes(uint8_t *state) {
37     for (int i = 0; i < 16; i++) {
38         state[i] = sbox[state[i]];
39     }
40 }
41
42 // Function to perform ShiftRows transformation
43 void shift_rows(uint8_t *state) {
44     uint8_t temp;
45
46     // Row 1: shift left by 1
47     temp = state[1];
48     state[1] = state[5];
49     state[5] = state[9];
50     state[9] = state[13];
51     state[13] = temp;
52
53     // Row 2: shift left by 2
54     temp = state[2];
55     state[2] = state[10];
56     state[10] = temp;
57     temp = state[6];
58     state[6] = state[14];
```

Aes encryption - C

Copy   

```
57     temp = state[6];
58     state[6] = state[14];
59     state[14] = temp;
60
61     // Row 3: shift left by 3
62     temp = state[15];
63     state[15] = state[11];
64     state[11] = state[7];
65     state[7] = state[3];
66     state[3] = temp;
67 }
68
69 // Galois field multiplication by 2
70 uint8_t gf_mul_2(uint8_t x) {
71     return (x << 1) ^ ((x >> 7) & 1) * 0x1b);
72 }
73
74 // Galois field multiplication by 3
75 uint8_t gf_mul_3(uint8_t x) {
76     return gf_mul_2(x) ^ x;
77 }
78
79 // Function to perform MixColumns transformation
80 void mix_columns(uint8_t *state) {
81     uint8_t temp[16];
82
83     for (int i = 0; i < 4; i++) {
84         int col = i * 4;
85         temp[col] = gf_mul_2(state[col]) ^ gf_mul_3(state[col + 1]) ^ state[col + 2] ^ state[col + 3];
86         temp[col + 1] = state[col] ^ gf_mul_2(state[col + 1]) ^ gf_mul_3(state[col + 2]) ^ state[col + 3];
87         temp[col + 2] = state[col] ^ state[col + 1] ^ gf_mul_2(state[col + 2]) ^ gf_mul_3(state[col + 3]);
88         temp[col + 3] = state[col] ^ state[col + 1] ^ state[col + 2] ^ gf_mul_2(state[col + 3]);
89     }
```

```
93
94 // Function to add round key
95 void add_round_key(uint8_t *state, uint8_t *round_key) {
96     for (int i = 0; i < 16; i++) {
97         state[i] ^= round_key[i];
98     }
99 }
.00
.01 // Key expansion function
.02 void key_expansion(uint8_t *key, uint8_t *round_keys) {
.03     uint8_t temp[4];
.04     int i = 0;
.05
.06     // Copy original key
.07     memcpy(round_keys, key, 16);
.08
.09     // Generate remaining round keys
.10     for (i = 4; i < 44; i++) {
.11         memcpy(temp, &round_keys[(i - 1) * 4], 4);
.12
.13         if (i % 4 == 0) {
.14             // RotWord
.15             uint8_t k = temp[0];
.16             temp[0] = temp[1];
.17             temp[1] = temp[2];
.18             temp[2] = temp[3];
.19             temp[3] = k;
.20
.21             // SubWord
.22             temp[0] = sbox[temp[0]];
```


Aes encryption · C

Copy



```
124         temp[2] = sbox[temp[2]];
125         temp[3] = sbox[temp[3]];
126
127         // XOR with round constant
128         temp[0] ^= rcon[i / 4];
129     }
130
131     // XOR with previous word
132     for (int j = 0; j < 4; j++) {
133         round_keys[i * 4 + j] = round_keys[(i - 4) * 4 + j] ^ temp[j];
134     }
135 }
136 }
137
138 // AES encryption function
139 void aes_encrypt(uint8_t *plaintext, uint8_t *key, uint8_t *ciphertext) {
140     uint8_t state[16];
141     uint8_t round_keys[176]; // 11 round keys * 16 bytes
142
143     // Copy plaintext to state
144     memcpy(state, plaintext, 16);
145
146     // Expand key
147     key_expansion(key, round_keys);
148
149     // Initial round
150     add_round_key(state, round_keys);
151
152     // Main rounds (9 rounds for AES-128)
153     for (int round = 1; round < NUM_ROUNDS; round++) {
```

Aes encryption - C

Copy

```
.52 // Main rounds (9 rounds for AES-128)
.53 for (int round = 1; round < NUM_ROUNDS; round++) {
.54     sub_bytes(state);
.55     shift_rows(state);
.56     mix_columns(state);
.57     add_round_key(state, &round_keys[round * 16]);
.58 }
.59
.60 // Final round (no MixColumns)
.61 sub_bytes(state);
.62 shift_rows(state);
.63 add_round_key(state, &round_keys[NUM_ROUNDS * 16]);
.64
.65 // Copy state to ciphertext
.66 memcpy(ciphertext, state, 16);
.67 }
.68
.69 // Helper function to print hex data
.70 void print_hex(const char *label, uint8_t *data, int len) {
.71     printf("%s: ", label);
.72     for (int i = 0; i < len; i++) {
.73         printf("%02x", data[i]);
.74     }
.75     printf("\n");
.76 }
.77
.78 int main() {
.79     // Example plaintext (16 bytes)
.80     uint8_t plaintext[16] = {
.81         0x32, 0x43, 0xf6, 0xa8, 0x88, 0x5a, 0x30, 0x8d,
```

Coding

the S-box

- **ShiftRows:** Cyclically shifts rows in the state array
- **MixColumns:** Mixes data within each column using Galois field arithmetic
- **AddRoundKey:** XORs the state with the round key
- **Key Expansion:** Generates 11 round keys from the original key

How it works:

1. The plaintext (16 bytes) goes through an initial round key addition
2. Then 9 main rounds of: SubBytes → ShiftRows → MixColumns → AddRoundKey
3. A final round without MixColumns
4. The result is the encrypted ciphertext

The example encrypts the test vector from

Reply...

+

Sonnet 4.5

↑

Claude is AI and can make mistakes. Please double-check responses.

Aes encryption - C

Copy

```
181     0x32, 0x43, 0xf6, 0xa8, 0x88, 0x5a, 0x30, 0x8d,
182     0x31, 0x31, 0x98, 0xa2, 0xe0, 0x37, 0x07, 0x34
183 };
184
185 // Example key (16 bytes for AES-128)
186 uint8_t key[16] = {
187     0x2b, 0x7e, 0x15, 0x16, 0x28, 0xae, 0xd2, 0xa6,
188     0xab, 0xf7, 0x15, 0x88, 0x09, 0xcf, 0x4f, 0x3c
189 };
190
191 // Buffer for ciphertext
192 uint8_t ciphertext[16];
193
194 printf("=== AES-128 Encryption Example ===\n\n");
195
196 // Print input
197 print_hex("Plaintext ", plaintext, 16);
198 print_hex("Key       ", key, 16);
199
200 // Perform encryption
201 aes_encrypt(plaintext, key, ciphertext);
202
203 // Print result
204 print_hex("Ciphertext", ciphertext, 16);
205
206 printf("\n=== Encryption Complete ===\n");
207
208 return 0;
209 }
```

4. debugging

debugging ▾ Share

```
#include<stdio.h>
int main(){
int x = 10
printf("Value: %d", x)
return 0;
}
Find the errors and correct the code claude.
```

6:26 pm ↺ ↻ 📄

I found two syntax errors in your C code:

Errors:

1. Missing semicolon after `int x = 10`
2. Missing semicolon after the `printf` statement

Corrected code:

```
c
#include<stdio.h>
int main(){
    int x = 10;
    printf("Value: %d", x);
    return 0;
}
```

Reply...
+ Sonnet 4.5 ▾ ⬆

Claude is AI and can make mistakes. Please double-check responses.

debugging ▾

1. Missing semicolon after `int x = 10`
2. Missing semicolon after the `printf` statement

Corrected code:

```
c
#include<stdio.h>
int main(){
    int x = 10;
    printf("Value: %d", x);
    return 0;
}
```

The code will now compile and run correctly, printing: `Value: 10`

📄 👍 🗨️ ↺

✳️

5. Translation

Translation ▾

Share

Translate the following into Hindi:
Artificial Intelligence is transforming industries worldwide.

Here's the Hindi translation:
कृत्रिम बुद्धिमत्ता दुनिया भर में उद्योगों को बदल रही है।
(Kṛtrim buddhimattā duniyā bhar meṁ udyogorṇ ko badal rahī hai.)





6. Table Creation

Table Creation ▾

Share

Create a comparison table between LSTM and Transformer models.

Here's a comprehensive comparison between LSTM and Transformer models:

Aspect	LSTM	Transformer
Architecture	Recurrent neural network with memory cells and gates (input, forget, output)	Attention-based architecture with encoder-decoder structure (or encoder/decoder only)
Processing	Sequential - processes tokens one at a time in order	Parallel - processes all tokens simultaneously
Training Speed	Slower due to sequential dependencies	Much faster due to parallelization
Long-range Dependencies	Struggles with very long sequences despite gates; information can degrade over time	Excellent at capturing long-range dependencies through self-attention mechanism
Memory	Hidden state and cell state carry information forward	Attention mechanism allows direct access to relevant parts of the input sequence

Table Creation

Share

Memory Mechanism	Hidden state and cell state carry information forward	Attention mechanism allows direct access to all previous tokens
Computational Complexity	$O(n)$ per layer where n is sequence length	$O(n^2)$ due to self-attention computing relationships between all token pairs
Positional Information	Inherent through sequential processing	Requires explicit positional encodings added to inputs
Scalability	Limited scalability to very large models	Highly scalable - powers models with billions of parameters
Interpretability	Gate activations provide some insight but limited	Attention weights offer clear visualization of token relationships
Use Cases	Time series, smaller NLP tasks, real-time streaming data	Large language models, translation, document understanding, any task requiring global context
Hardware Efficiency	Less efficient on modern GPUs/TPUs	Highly optimized for parallel hardware architectures
Gradient Flow	Better than vanilla RNNs but still prone to vanishing gradients	Residual connections and layer normalization provide stable gradients

Reply...

+

Sonnet 4.5

↑

Claude is AI and can make mistakes. Please double-check responses.

7Drafting Email

Write a professional email requesting permission to submit a project report one day late.

I'll draft a professional email for you requesting a one-day extension on your project report.

Subject: Request for One-Day Extension on Project Report Submission

Dear [Professor/Manager's Name],

I hope this message finds you well. I am writing to request permission to submit the [project name/course name] report one day past the original deadline of [original due date].

I have been working diligently on this report and have made substantial progress. However, I need additional time to ensure the quality and thoroughness of my analysis meet the standards expected for this assignment. An extension until [new date] would allow me to properly review and refine my work.

I understand the importance of meeting deadlines and apologize for any inconvenience this may cause. I am committed to submitting a comprehensive and well-prepared report by the requested date.



I hope this message finds you well. I am writing to request permission to submit the [project name/course name] report one day past the original deadline of [original due date].

I have been working diligently on this report and have made substantial progress. However, I need additional time to ensure the quality and thoroughness of my analysis meet the standards expected for this assignment. An extension until [new date] would allow me to properly review and refine my work.

I understand the importance of meeting deadlines and apologize for any inconvenience this may cause. I am committed to submitting a comprehensive and well-prepared report by the requested date.

Thank you for considering my request. I would be happy to discuss this further if you have any questions or concerns.

Best regards,

[Your Name]

[Your Contact Information]

Feel free to adjust the tone or add specific details about your circumstances if needed. The key is to be respectful, take responsibility, and show that you're committed to delivering quality work.

8. Idea Generation

Idea Generation

Generate 5 innovative AI project ideas for final year engineering students.

Here are 5 innovative AI project ideas suitable for final year engineering students:

1. Smart Campus Navigation System with Real-Time Crowd Analysis

Build an AI-powered mobile app that helps students navigate campus while avoiding crowded areas. The system would use computer vision to analyze camera feeds (with privacy considerations), predict crowd density in different locations like libraries, cafeterias, and lecture halls, and suggest optimal routes. You could incorporate reinforcement learning for route optimization and time-series forecasting for predicting busy periods based on class schedules and historical data.

2. AI-Powered Mental Health Companion for Students

Create a conversational AI system that helps students manage stress and mental well-being through daily check-ins, mood tracking, and personalized coping strategies. The system could use natural language processing to detect emotional states from text, provide evidence-based cognitive behavioral therapy techniques, and alert students when patterns suggest they should seek professional help. This addresses a real need in academic environments while involving sentiment analysis, dialogue systems, and personalized

Idea Generation

recommendation algorithms.

3. Automated Code Review Assistant with Vulnerability Detection

Develop an AI tool that reviews code submissions, identifies bugs, suggests improvements, and detects security vulnerabilities. Using large language models fine-tuned on code repositories and static analysis tools, the system could provide explanations for issues it finds and suggest fixes. This could be particularly useful for teaching environments and would involve working with transformer models, code understanding, and potentially graph neural networks for analyzing code structure.

4. Agricultural Crop Disease Detection Using Drone Imagery

Build a system that uses computer vision and drone imagery to detect crop diseases early. The AI would analyze multispectral images to identify patterns invisible to the naked eye, classify different disease types, and provide treatment recommendations. You could create a mobile app for farmers that works with affordable drones or smartphones. This combines image classification, object detection, and practical real-world impact, especially relevant for agricultural communities.

5. Real-Time Sign Language Translation System

Create an AI system that translates sign language into text or speech in real-time using computer vision and gesture recognition. Using hand tracking, pose estimation, and sequence modeling (like LSTMs or Transformers), the system could recognize sign

Idea Generation

Share

a mobile app for farmers that works with affordable drones or smartphones. This combines image classification, object detection, and practical real-world impact, especially relevant for agricultural communities.

5. Real-Time Sign Language Translation System

Create an AI system that translates sign language into text or speech in real-time using computer vision and gesture recognition. Using hand tracking, pose estimation, and sequence modeling (like LSTMs or Transformers), the system could recognize sign language gestures from webcam feeds and convert them to spoken or written language. This has significant accessibility implications and involves challenging problems in action recognition and temporal modeling.

Each of these projects balances technical challenge with practical impact, uses current AI techniques, and can be scoped appropriately for a final year project. Would you like me to elaborate on any of these ideas or discuss implementation approaches?





9 Q&A

Q&A

Share

Generate 5 viva questions with answers on AES encryption.

Here are 5 viva questions with answers on AES encryption:

- 1. What is AES and why was it developed?**

AES (Advanced Encryption Standard) is a symmetric block cipher that encrypts data in 128-bit blocks. It was developed to replace the aging DES (Data Encryption Standard) which had become vulnerable due to its small 56-bit key size. In 1997, NIST announced a competition to find a replacement, and the Rijndael cipher designed by Belgian cryptographers Joan Daemen and Vincent Rijmen was selected in 2001 and standardized as AES. It's now the most widely used encryption standard globally, securing everything from wireless networks to financial transactions.
- 2. Explain the different key sizes in AES and how they affect security.**

AES supports three key sizes: 128-bit, 192-bit, and 256-bit. The key size directly affects the number of transformation rounds performed during encryption. AES-128 uses 10 rounds, AES-192 uses 12 rounds, and AES-256 uses 14 rounds. While larger key sizes provide theoretically stronger security against brute-force attacks (2^{128} , 2^{192} , and 2^{256} possible keys respectively), AES-128 is still considered secure for most applications. The choice often balances security requirements against performance, as more rounds mean

Reply...



Sonnet 4.5 

Claude is AI and can make mistakes. Please double-check responses.

slower encryption/decryption.

3. Describe the four main operations in each AES encryption round.

Each AES round (except the final round) consists of four operations:

- **SubBytes:** A non-linear substitution step where each byte is replaced with another according to a lookup table (S-box), providing confusion
- **ShiftRows:** A transposition step where rows of the state array are cyclically shifted by different offsets
- **MixColumns:** A mixing operation that operates on columns of the state, combining the four bytes in each column (skipped in the final round)
- **AddRoundKey:** Each byte of the state is XORed with a byte of the round key derived from the cipher key

These operations together provide both confusion and diffusion properties essential for strong encryption.

4. What is the difference between AES encryption modes like ECB, CBC, and CTR?

These are modes of operation that define how AES processes data larger than a single block:

- **ECB (Electronic Codebook):** Each block is encrypted independently with the same

Reply...

+

Sonnet 4.5 ▾

|||

4. What is the difference between AES encryption modes like ECB, CBC, and CTR?

These are modes of operation that define how AES processes data larger than a single block:

- **ECB (Electronic Codebook):** Each block is encrypted independently with the same key. This is insecure for most uses as identical plaintext blocks produce identical ciphertext blocks, revealing patterns.
- **CBC (Cipher Block Chaining):** Each plaintext block is XORed with the previous ciphertext block before encryption, using an initialization vector (IV) for the first block. This prevents pattern recognition but requires sequential processing.
- **CTR (Counter):** Converts AES into a stream cipher by encrypting a counter value and XORing with plaintext. It allows parallel processing and random access but requires careful IV/counter management.

5. How does AES key expansion work and why is it necessary?

Key expansion (or key schedule) is the process of generating round keys from the original cipher key. For AES-128, the 128-bit key is expanded into 11 round keys (one for the initial AddRoundKey and one for each of the 10 rounds). The expansion uses a combination of operations including rotation, S-box substitution, and XOR with round constants (Rcon). This is necessary because using the same key repeatedly would be cryptographically weak.

10 Data formatting

Data Formatting

Convert the following information into JSON format:
Name: Shobana S
Department: Computer Science in artificial intelligence
Skills: Python,java,SQL,C
Experience: 2 year

```
json
{
  "name": "Shobana S",
  "department": "Computer Science in artificial intelligence",
  "skills": ["Python", "Java", "SQL", "C"],
  "experience": "2 years"
}
```

Pin cone



2025-04

Search...

Ctrl K

Status

Support

Log In

Sign up free

Guides

Reference

Examples

Models

Integrations

Troubleshooting

Releases

Inference

Embed

Rerank

Models

Admin

API keys

Create an API key

List API keys

Get API key details

Update an API key

Delete an API key

Projects

API KEYS

Create an API key

Create a new API key for a project. Developers can use the API key to authenticate requests to Pinecone's Data Plane and Control Plane APIs.

POST /admin/projects/{project_id}/api-keys

Authorizations

Authorization string header required

An [access token](#) must be provided in the Authorization header using the Bearer scheme.

Path Parameters

curl CLI

PINECONE_PROJECT_ID="32c8235a-5220-4a80-a9f1-69c24
PINECONE_ACCESS_TOKEN="YOUR_ACCESS_TOKEN"

curl -X POST "https://api.pinecone.io/admin/project
-H "X-Pinecone-API-Version: 2025-04" \
-H "Authorization: Bearer \$PINECONE_ACCESS_TO
-d '{
 "name": "example-api-key",
 "roles": ["ProjectEditor"]
'

curl CLI

{
 "key": {
 "id": "62b0dbfe-3489-4b79-b850-34d911527c88",
 "name": "example-api-key",
 "project_id": "32c8235a-5220-4a80-a9f1-69c2410
 "roles": [
 "ProjectEditor"
],
 "created_at": "2025-10-20T23:40:27.069075Z"
 }
}

API key generated

Your "**default**" API key has been generated.

*****_****_****_****_*****



You must copy and save this key immediately

For security reasons, we will not show you this key again after you close this dialog.
Please save it somewhere secure for future use.

Close

Claude Code needs a Pinecone API key for secure access to your project.

+ API key